Sylf AI Planner Modes

Product Requirements Document (PRD)

Document status	Draft for product, design, and engineering review
Version	1.0
Date	24 September 2026
Product	Sylf
Feature	AI Planner Modes
Primary audience	Product, Design, Frontend, Backend, AI/Prompt Engineering, QA

A planning system that converts academic goals into structured, adaptive Sylf quests - without becoming a generic timetable generator.

Internal product specification. Implementation details are recommendations unless explicitly marked as acceptance criteria.

Contents

1 Executive Summary **5**

2 Feature Overview **5**
2.1 Product concept . . . 5
2.2 Why modes are necessary . . . 5
2.3 Product positioning . . . 6

3 Problem Statement **6**

4 Goals and Non-Goals **6**
4.1 Goals . . . 6
4.2 Non-goals for the initial release . . . 6

5 Target Users and Personas **7**

6 Primary Use Cases **7**

7 User Stories **7**

8 Planner Modes **8**
8.1 Mode A - Exam Planner . . . 8
8.1.1 Example Exam Planner input . . . 9
8.1.2 Example generated plan excerpt . . . 9
8.2 Mode B - Syllabus / Course Roadmap . . . 9
8.2.1 Example roadmap . . . 9
8.3 Mode C - Chapter / Unit Mastery . . . 10

9 Input Requirements **10**
9.1 Shared input model . . . 10
9.2 Progressive disclosure . . . 11

10 Output Requirements **11**
10.1 Shared output fields . . . 11
10.2 Review before commit . . . 11

11 Planning Logic **12**
11.1 High-level pipeline . . . 12
11.2 Normalization . . . 12
11.3 Workload estimation . . . 12
11.4 Exam stage allocation . . . 12
11.5 Roadmap allocation . . . 12
11.6 Mastery allocation . . . 12

12 Quest-Generation Behavior **12**
12.1 Quest creation principles . . . 12

12.2 Recommended quest object example . . . 13

13 Adaptive Planning and Rebalancing 13

13.1 Triggers . . . 13
13.2 Rebalancing rules . . . 13
13.3 Replan example . . . 14

14 UX Flow 14

14.1 Entry points . . . 14
14.2 Core creation flow . . . 14
14.3 Design principles . . . 14

15 Edge Cases 14

16 Validation Rules 15

16.1 Input validation . . . 15
16.2 Plan validation . . . 15

17 Data and State Requirements 16

17.1 Recommended conceptual entities . . . 16
17.2 State model . . . 16

18 Frontend Requirements 16

19 Backend and API Requirements 16

19.1 Principles . . . 16
19.2 Recommended API shape . . . 17
19.3 Commit integrity . . . 17

20 AI Prompting and Orchestration Requirements 17

20.1 Orchestration stages . . . 17
20.2 Prompting principles . . . 17
20.3 Model output reliability . . . 18

21 Integration with Existing Sylf Features 18

21.1 Quests . . . 18
21.2 Focus sessions . . . 18
21.3 Progress tracking . . . 18
21.4 Streaks . . . 18
21.5 System / AI assistant . . . 18
21.6 Future adaptive planning . . . 18

22 Safety and Reliability 18

23 Performance Requirements 19

24 Analytics and Telemetry 19

25 Accessibility Requirements **19**

26 Functional Requirements Summary **20**

27 Acceptance Criteria **20**

27.1 Mode-level acceptance 20

27.2 Shared acceptance 20

28 Phased Implementation Roadmap **21**

28.1 Phase 0 - Architecture and contracts 21

28.2 Phase 1 - MVP generation and review 21

28.3 Phase 2 - Rebalancing and deeper integration 21

28.4 Phase 3 - Adaptive intelligence 21

29 Open Product Decisions **21**

30 Technical Risks **22**

31 Recommended MVP Scope **22**

32 Phase 2 / Future Enhancements **23**

33 Appendix A - Example End-to-End Exam Flow **23**

34 Appendix B - Product Decision Principles **24**

1 Executive Summary

AI Planner Modes extends Sylf's existing AI planning capability into a structured academic planning system built around distinct user intents. Instead of presenting one generic "make me a study plan" experience, Sylf asks the user what they are trying to accomplish and then applies a planning model appropriate to that goal.

The initial release defines three planning modes:

- **Exam Planner**: deadline-driven, day-level planning for a specific exam or test.
- **Syllabus / Course Roadmap**: long-horizon planning for completing a course, qualification, or syllabus, primarily at month/week granularity.
- **Chapter / Unit Mastery**: short-horizon mastery planning for one topic, chapter, or unit.

All modes share a common lifecycle:

Shared planning loop

Plan → Complete quests → Sylf tracks progress → Rebalance / replan → Updated plan

The key product principle is that planning should feel guided, lightweight, and academically aware. Users should not be required to fill in a large form. Sylf should gather only the minimum necessary information, progressively disclose optional controls, infer reasonable structure where safe, and show the proposed plan before committing it to the user's quest system.

The MVP should prioritize trustworthy plan generation, review/edit before commit, integration with existing Quests and Focus workflows, deterministic validation, and safe rebalancing. More advanced adaptation - such as mastery inference, performance-based difficulty adjustment, document parsing, and institution-specific exam strategies - belongs to later phases.

2 Feature Overview

2.1 Product concept

AI Planner Modes is a goal-oriented planning layer inside Sylf. It converts an academic objective into a structured plan and, where appropriate, into executable Sylf quests. The planner is not intended to replace a teacher, course specification, or exam board. It organizes user-provided academic scope into a practical sequence of work.

2.2 Why modes are necessary

A student preparing for an exam has a fundamentally different scheduling problem from a student trying to finish an entire syllabus over six months. Treating both requests as the same prompt produces plans that are either too granular, too vague, or poorly matched to the user's horizon.

Each mode therefore defines its own:

- required inputs;
- planning horizon and granularity;
- sequencing logic;
- output structure;
- quest types;
- adaptation rules;
- review and commitment flow.

2.3 Product positioning

The planner should feel native to Sylf because its output connects directly to Sylf concepts: Quests, Focus sessions, progress, streaks, System/AI, and future adaptive planning. Its value is not merely that an LLM can generate a timetable; its value is that the plan becomes an actionable, trackable system inside the product.

3 Problem Statement

Students commonly know *what* they must study but struggle to determine *how to distribute the work*, *what to do next*, and *how to recover when the plan changes*. Existing approaches have recurring problems:

- generic AI study plans often ignore actual availability, confidence, or exam structure;
- manually creating dozens of quests is tedious and discourages planning;
- long-range schedules become inaccurate when generated at daily granularity months in advance;
- missed days cause plans to accumulate overdue tasks rather than adapt;
- students frequently over-allocate content coverage and under-allocate practice, recall, and mistake review;
- planning tools often exist separately from the execution environment, forcing the user to copy tasks into another app.

Sylf can address this by creating planning modes that understand the user's time horizon, generate the correct level of detail, and connect directly to the user's study execution loop.

4 Goals and Non-Goals

4.1 Goals

1. Help users turn a concrete academic goal into a usable plan in a few guided steps.
2. Generate plans whose granularity matches the planning horizon.
3. Ensure exam plans contain meaningful learning, consolidation, practice, and readiness phases rather than only content coverage.
4. Convert approved plans into Sylf quests without duplicating planner state or backend business logic.
5. Allow users to rebalance a plan after missed work, changed availability, or changed progress.
6. Preserve user agency: generated plans are proposals, not irreversible schedules.
7. Integrate planning with Focus sessions and progress tracking so the plan remains useful after generation.
8. Create a stable data and orchestration foundation for future adaptive planning.

4.2 Non-goals for the initial release

- Automatically determining an official syllabus from the subject name alone without user confirmation.
- Guaranteeing exam results, grades, readiness, or mastery.
- Replacing teachers, tutors, school timetables, or official exam-board guidance.
- Fully autonomous rescheduling without user visibility or approval.
- Generating or grading large banks of assessment questions in the MVP.
- Building a full calendar product inside the planner.
- Scheduling every day of a multi-month course roadmap at generation time.
- Inferring medical, psychological, or disability-related study capacity.

5 Target Users and Personas

Persona	Primary mode	Need	Success condition
Exam-focused student	Exam Planner	Has a fixed exam date and syllabus but does not know how to distribute revision and practice.	Opens Sylf and knows exactly what to study today while still preserving time for practice and final review.
Long-horizon learner	Syllabus Roadmap	Needs to complete an entire course or qualification over several months.	Sees realistic monthly/weekly milestones without being overwhelmed by hundreds of premature daily tasks.
Topic-recovery student	Unit Mastery	Needs to fix one weak chapter or learn one unit efficiently.	Receives a short sequence that moves from understanding to recall, practice, mistakes, and mastery check.
Busy / inconsistent student	All modes	Availability changes and missed days make static schedules fail.	Can rebalance remaining work without losing completed progress or manually rebuilding the plan.

6 Primary Use Cases

1. A student has an exam in 18 days and pastes the topics included. Sylf creates a day-level preparation plan with coverage, revision, practice, and exam-readiness stages.
2. A student wants to complete A-Level Physics by a target month. Sylf creates a monthly roadmap, then later converts one selected month into detailed quests.
3. A student wants to master “Electric Fields” before next week. Sylf creates a compact mastery sequence and optionally turns each stage into quests.
4. A user misses two planned study days. Sylf proposes a rebalanced plan that protects the exam deadline and buffer time rather than simply moving every task forward.
5. A user’s availability changes. Sylf recalculates remaining workload and clearly explains what changed.
6. A user completes a quest early or marks a topic as already confident. Sylf can reduce redundant coverage and allocate more time to practice or weak areas.

7 User Stories

ID	User story	Acceptance signal
US-01	As a student, I want to choose what type of planning problem I have so the planner asks relevant questions.	Mode selection changes the input flow and output granularity.
US-02	As an exam student, I want to enter my exam date and syllabus so Sylf can work backwards from the deadline.	Generated plan respects the deadline and contains multiple preparation phases.
US-03	As a course learner, I want a long-term roadmap rather than hundreds of daily tasks.	Course mode produces milestone-based monthly/weekly output by default.
US-04	As a student, I want to review and edit the plan before it creates quests.	No plan is committed until user confirmation.

US-05	As a user who missed work, I want Sylf to rebalance the remaining plan.	Completed work is preserved; only future/uncompleted items are rescheduled unless explicitly chosen otherwise.
US-06	As a user, I want my plan to connect to Focus so I can execute the next quest immediately.	Eligible quests can launch or link to a Focus session using existing product flows.
US-07	As a user, I want the planner to explain why it scheduled a topic.	Plan items can expose concise rationale without showing chain-of-thought.
US-08	As a user, I want to adjust availability later.	Replan preview shows workload changes before applying them.

8 Planner Modes

8.1 Mode A - Exam Planner

Purpose: prepare for a fixed assessment by creating a day-level plan that works backward from the exam date.

Typical inputs:

- subject and exam/test name;
- exam date (required);
- syllabus/topics/chapters included (required);
- current progress or confidence by topic (optional but strongly encouraged);
- available days/hours or existing Sylf availability profile;
- optional exam format: multiple-choice, structured response, essays, practical, oral, mixed;
- optional constraints: no-study days, school schedule, preferred rest day, maximum daily workload.

Planning stages:

Coverage → Consolidation → Practice → Exam Readiness

These stages are conceptual. They may overlap when the deadline is short or when the student is already familiar with some material.

Quest types:

- Learn
- Review
- Recall
- Practice
- Timed Practice
- Mistake Review
- Mock Exam
- Final Review

Buffer principle: the planner should not fill 100% of available capacity. It should preserve recovery space, especially near high-stakes deadlines. Buffer may be represented as unallocated capacity or explicit catch-up/light-review periods.

8.1.1 Example Exam Planner input

Example input

Subject: A-Level Physics
Exam: Paper 4 Mock
Date: 18 days from today
Included topics: Electric fields, capacitors, magnetic fields, electromagnetic induction
Current confidence: Electric fields 2/5, capacitors 3/5, magnetic fields 1/5, induction 2/5
Availability: 90 min weekdays, 3 hours Saturday, Sunday unavailable
Format: structured calculation and explanation questions

8.1.2 Example generated plan excerpt

Day	Stage	Quest	Type	Rationale
1	Coverage	Electric fields fundamentals	Learn	Weak confidence and prerequisite for later questions.
2	Coverage	Electric field calculations	Practice	Converts concepts into exam-style application.
3	Coverage	Magnetic fields fundamentals	Learn	Lowest-confidence major topic.
4	Consolidation	Recall: electric + magnetic fields	Recall	Early retrieval before moving deeper.
5	Coverage	Capacitors and energy	Learn	Medium-confidence topic; shorter allocation.
6	Practice	Mixed field questions	Practice	Interleaves related concepts.
7	Buffer	Catch-up or light review	Review	Protects plan against missed work.

8.2 Mode B - Syllabus / Course Roadmap

Purpose: create a long-horizon learning roadmap for an entire course, qualification, or broad syllabus.

Typical inputs:

- course/subject;
- qualification, grade, year, or curriculum label;
- syllabus/units/chapters;
- target completion date;
- current completion/progress;
- approximate weekly availability;
- optional constraints: school terms, holidays, external deadlines, difficult units.

Default output granularity: monthly or weekly milestones - not daily quests across the entire horizon.

Expansion behavior: the user can select a milestone and choose *Plan this month* or *Break this down*, converting that segment into a detailed weekly/daily quest plan using the same underlying planning engine.

8.2.1 Example roadmap

Period	Coverage	Milestone	Checkpoint

September	Measurements, kinematics, dynamics	Mechanics foundation complete	Short cumulative review
October	Forces, work/energy/power, momentum	Core mechanics complete	Mixed mechanics questions
November	Materials, waves, superposition	Waves block complete	Recall + topic questions
December	Electricity + consolidation	First-term scope consolidated	Progress review and roadmap adjustment

8.3 Mode C - Chapter / Unit Mastery

Purpose: create a compact learning sequence for one chapter, unit, or topic.

Typical inputs:

- subject;
- chapter/topic/unit;
- optional subtopics;
- optional deadline;
- current confidence/progress;
- available study time.

Default sequence:

Understand → Learn subtopics → Active Recall → Practice → Review Mistakes → Mastery Check

The sequence may compress or skip stages when the user indicates existing competence, but the planner should not remove practice or mastery verification solely to fit more content.

9 Input Requirements

9.1 Shared input model

Field	Required?	Applies to	Notes
Mode	Yes	All	Exam, Roadmap, or Mastery.
Subject / course	Yes	All	Free text or existing Sylf subject where available.
Scope	Yes	All	Topics, chapters, syllabus units, or pasted outline.
Deadline / target date	Exam: Yes; others: Optional/Recommended	All	Must be future-dated where required.
Availability	Recommended	All	Can be simple hours/week, per-day availability, or existing profile.
Current progress	Optional	All	Not started / some covered / mostly covered, or per-topic values.
Confidence	Optional	Exam/Mastery	Coarse scale is sufficient; do not require detailed self-assessment.
Exam format	Optional	Exam	Used to influence practice types, not to infer official rules.

Constraints	Optional	All	Rest days, no-study dates, maximum session length, etc.

9.2 Progressive disclosure

The planner should avoid a single long form. A recommended flow is 4-6 lightweight steps with optional controls collapsed behind “More options”.

Example guided flow - Exam Planner

Step 1: What are you preparing for?
Step 2: When is it?
Step 3: What is included? (type, paste, select existing subject, or future upload)
Step 4: How prepared are you?
Step 5: When can you study?
Step 6: Review inferred assumptions and generate plan

10 Output Requirements

Every generated plan should return a structured machine-readable plan plus a user-facing explanation. The UI should not rely on parsing free-form prose to create quests.

10.1 Shared output fields

- plan title and mode;
- source scope summary;
- planning horizon;
- assumptions used;
- workload summary;
- plan items / milestones;
- quest type per actionable item;
- duration estimate or effort band;
- optional rationale suitable for UI display;
- buffers / recovery capacity;
- warnings or feasibility flags;
- version/revision metadata.

10.2 Review before commit

Generated plans must first exist as a **draft**. The user can:

- edit titles, dates, durations, and ordering;
- lock important items;
- remove items;
- ask Sylf to regenerate a portion;
- approve the plan and create quests;
- cancel without mutating existing quest state.

11 Planning Logic

11.1 High-level pipeline

Collect inputs → Normalize scope → Estimate workload → Allocate stages → Schedule → Validate → Preview

11.2 Normalization

The planner should normalize user input into topic units. A pasted syllabus may contain headings, subheadings, and notes; the normalized representation should preserve the user's wording while separating schedulable units from metadata. The system should not silently invent missing official syllabus content.

11.3 Workload estimation

Workload estimates should use coarse, explainable factors rather than pretend to know exact study time. Suggested factors:

- topic breadth;
- current progress;
- confidence;
- user availability;
- time until deadline;
- selected mode;
- assessment format where supplied.

If the plan is infeasible, Sylf should say so and offer tradeoffs: reduce scope, increase availability, extend deadline (where possible), or choose a lower-confidence target such as "coverage only" rather than silently overloading the calendar.

11.4 Exam stage allocation

A suggested policy is to reserve explicit capacity for practice and final review before allocating all available time to content coverage. Exact percentages should not be hard-coded globally; they may vary based on deadline, progress, and exam format. However, the validator should detect plans with zero practice or zero consolidation unless there is a clear user-driven reason.

11.5 Roadmap allocation

Roadmaps should use milestones sized to the user's horizon. The algorithm should prefer coherent topic blocks and prerequisites over equal-sized calendar chunks. It should include periodic consolidation checkpoints and should not create distant daily quests until the user expands a milestone.

11.6 Mastery allocation

Mastery plans should be short and sequential, but allow interleaving where useful. A mastery check should remain at the end even if earlier tasks are completed quickly.

12 Quest-Generation Behavior

12.1 Quest creation principles

- One quest should represent a meaningful unit of work, not an entire course chapter if that chapter cannot reasonably fit in one session.

- Quest titles should be concise and action-oriented.
- Quest type should communicate intent: Learn, Recall, Practice, etc.
- Estimated duration should be bounded by user preferences and availability.
- Quest metadata should retain a link to its source planner plan and plan item.
- Planner-created quests should be distinguishable from manually created quests without creating a separate quest system.
- Replanning must not duplicate already-created or completed quests.

12.2 Recommended quest object example

```
{
  "planId": "plan_...",
  "planItemId": "item_...",
  "type": "PRACTICE",
  "title": "Mixed electric field questions",
  "subject": "A-Level Physics",
  "scheduledDate": "2026-10-05",
  "estimatedMinutes": 50,
  "topicRefs": ["electric-fields"],
  "stage": "PRACTICE",
  "locked": false,
  "plannerRevision": 3
}
```

This is illustrative, not a mandatory implementation schema.

13 Adaptive Planning and Rebalancing

13.1 Triggers

A replan may be initiated when:

- one or more planner-linked quests are missed;
- the user changes availability;
- the exam/target date changes;
- the user marks topics complete or changes confidence;
- the user explicitly requests “Rebalance plan”;
- a roadmap milestone is expanded into a more detailed plan.

13.2 Rebalancing rules

1. Never alter completed quest history.
2. Treat user-locked quests as fixed unless the user explicitly unlocks them.
3. Do not move work into the past.
4. Preserve the target deadline when feasible.
5. Preserve buffer/recovery capacity where feasible; if buffer must be consumed, state this clearly.
6. Do not create duplicate quest content because an item was moved.
7. Prefer redistributing unfinished work across remaining capacity rather than stacking multiple large tasks onto the next day.
8. Show a diff/summary before applying changes: moved, shortened, merged, removed, added.

13.3 Replan example

Example

The user misses Tuesday's "Magnetic fields fundamentals" quest. Instead of marking the plan broken, Sylf proposes: move magnetic fields to Wednesday, move the smaller capacitor review to Thursday, retain Saturday's mixed-practice block, and use half of Friday's buffer. The user can approve, edit, or keep the original plan.

14 UX Flow

14.1 Entry points

Recommended entry points:

- Planner home / System Hub: "Create a study plan";
- Quests: "Generate from AI Planner";
- subject/course context: "Plan this subject";
- existing roadmap: "Break down this milestone";
- plan detail: "Rebalance".

14.2 Core creation flow

1. **Choose mode** - three clear outcome-based options.
2. **Provide goal context** - subject/course, deadline, scope.
3. **Set availability** - reuse existing profile where possible, allow quick overrides.
4. **Optional refinement** - confidence, exam format, constraints.
5. **Generate** - show progress states tied to real orchestration steps.
6. **Review draft** - timeline/roadmap with editable items and assumptions.
7. **Commit** - create quests/milestones transactionally.
8. **Track** - plan detail shows completion and next recommended action.

14.3 Design principles

- Progressive disclosure over giant forms.
- Visual planning output first; explanatory prose second.
- Make assumptions visible but concise.
- Users must be able to change dates and workload without starting over.
- A single "next best action" should be obvious after plan creation.
- Mobile may use a different layout from desktop while preserving the same planning semantics.

15 Edge Cases

Edge case	Required behavior
Exam is tomorrow with a large syllabus	Flag infeasibility; create a triage plan if user chooses, prioritizing high-value review/practice rather than pretending full coverage is possible.

No availability provided	Use a clearly stated conservative default or ask one lightweight follow-up; never assume unlimited time.
Deadline in the past	Reject and ask for a valid future date, or offer “No fixed deadline” in modes where allowed.
Duplicate topics in pasted scope	Normalize duplicates while preserving user wording; show normalized scope for confirmation if changes are material.
Very large syllabus	Chunk processing; summarize/normalize first; ask for confirmation before detailed planning.
User already completed half the syllabus	Do not allocate full learning blocks to completed topics; schedule review/practice as appropriate.
User misses multiple days	Rebalance future items and surface feasibility risk; do not silently compress beyond daily workload limits.
User changes timezone	Preserve intended local study dates using canonical timezone handling.
AI returns malformed structure	Validate and retry server-side; never commit partially parsed output.
Quest creation partially fails	Use transaction/idempotency so users do not receive duplicate or half-created plans.
User deletes a planner-created quest	Mark corresponding plan item as user-modified/deleted; do not silently recreate it on next replan.

16 Validation Rules

16.1 Input validation

- Required fields must be validated before AI invocation.
- Exam date must be future-dated and timezone-aware.
- Availability must be non-negative and bounded to realistic per-day values.
- Scope must contain at least one schedulable topic/unit.
- Text inputs require sensible length limits and server-side enforcement.
- Unsupported or ambiguous mode combinations should be resolved before generation.

16.2 Plan validation

Before presenting a draft, deterministic validators should check:

- dates fall within the planning horizon;
- no duplicate plan item IDs;
- total scheduled effort does not exceed allowed capacity beyond configured tolerance;
- all required scope units are represented or explicitly excluded;
- exam plans contain appropriate non-coverage activity where feasible;
- no item is scheduled on a blocked day;
- roadmap output is not accidentally expanded to daily granularity;
- quest types are valid enumerated values;
- generated durations remain within allowed bounds;
- plan references and revision numbers are internally consistent.

17 Data and State Requirements

17.1 Recommended conceptual entities

Entity	Purpose
PlannerPlan	Top-level user plan: mode, title, horizon, status, assumptions, current revision.
PlannerScopeItem	Normalized topic/unit supplied or confirmed by the user.
PlannerMilestone	Long-horizon grouping, especially for course roadmap mode.
PlannerItem	Schedulable unit that may map to a quest.
PlannerRevision	Immutable snapshot/diff metadata for generation and rebalance history.
AvailabilitySnapshot	Availability inputs used for a particular revision.
Quest link	Existing quest reference to preserve one source of truth for quest execution.

17.2 State model

Suggested plan states:



REBALANCING is an operation/revision state, not a duplicate plan.

A plan should remain the stable parent object across revisions. Rebalancing should create a new revision rather than cloning an unrelated plan and losing history.

18 Frontend Requirements

ID	Requirement
FE-01	Provide an outcome-based mode selector with concise explanations.
FE-02	Use progressive disclosure; optional fields must not overwhelm the default path.
FE-03	Support a review/edit screen before any quests are committed.
FE-04	Represent exam plans as a timeline/day view and roadmaps as milestone-oriented month/week views.
FE-05	Show planning assumptions and feasibility warnings clearly but compactly.
FE-06	Expose rebalancing as a previewable action with a concise change summary.
FE-07	Use existing Sylf theme, mode, accessibility, and responsive systems.
FE-08	Loading UI must reflect actual orchestration stages rather than fabricated progress.
FE-09	Draft edits must not mutate production quests until commit succeeds.
FE-10	All relevant empty, loading, error, partial, and retry states must be designed.

19 Backend and API Requirements

19.1 Principles

- The server owns authorization, validation, AI orchestration, plan persistence, and quest commit integrity.
- The client never receives provider secrets or unrestricted AI credentials.
- Generation and commit are separate operations.
- Expensive generation endpoints should support request idempotency and appropriate rate/quota controls.
- Rebalance operations should reference the current plan revision to prevent lost updates.

19.2 Recommended API shape

The exact route naming should follow Sylf's existing conventions. A conceptual surface may include:

Operation	Method	Purpose
/planner/generate	POST	Validate inputs, orchestrate AI, run deterministic validators, return/persist draft revision.
/planner/:id	GET	Retrieve authorized plan with current revision and linked quest state.
/planner/:id/commit	POST	Transactionally create/update linked quests from approved draft.
/planner/:id/rebalance	POST	Generate a new revision based on progress/availability changes.
/planner/:id/revisions	GET	Optional revision history / audit data.
/planner/:id/archive	POST	Archive without deleting completed quest history.

19.3 Commit integrity

Quest creation should be idempotent. A retry after network failure must not create duplicate quests. If multiple quest records are created from one plan revision, use the application's existing transactional patterns so the user does not end up with a partially committed plan.

20 AI Prompting and Orchestration Requirements

20.1 Orchestration stages

A recommended pipeline is:

1. Input validation and authorization.
2. Scope normalization / topic extraction.
3. Deterministic calculation of available dates/capacity.
4. AI planning request using normalized scope and bounded constraints.
5. Structured-output schema validation.
6. Deterministic plan validation and repair/retry if needed.
7. Save draft revision.
8. Return user-facing draft and concise rationale.

20.2 Prompting principles

- Treat user-provided syllabus scope as authoritative unless the user asks for external enrichment.
- Require structured output rather than free-form prose as the system of record.
- Supply exact availability and scheduling constraints from deterministic code.
- Instruct the model not to schedule outside the horizon or on blocked dates.
- Ask for concise user-facing rationale, not hidden reasoning or chain-of-thought.
- Separate mode-specific instructions from shared planner rules.
- Include a feasibility field so the model can signal when the requested plan cannot reasonably fit.
- Never let the model invent persisted IDs, authorization decisions, or quota state.

20.3 Model output reliability

The backend must validate model output against a strict schema. Invalid enum values, missing dates, impossible durations, or malformed topic references should trigger repair/retry rather than reaching the client as partially trusted data.

21 Integration with Existing Sylf Features

21.1 Quests

Planner actions should create ordinary Sylf quests with planner metadata, not a parallel task system. Existing quest completion behavior remains authoritative.

21.2 Focus sessions

Planner-created quests should be launchable into the existing Focus experience. Completed Focus sessions can contribute to quest progress according to current quest behavior. The planner should not create a separate timer or session implementation.

21.3 Progress tracking

Plan progress should derive from linked quest/milestone state. Avoid maintaining a second manually synchronized completion percentage if the data can be derived reliably.

21.4 Streaks

Planner completion may naturally contribute to existing study activity and streak systems through normal quest/Focus behavior. The planner should not create its own competing streak mechanic in the MVP.

21.5 System / AI assistant

System can serve as the conversational surface for actions such as “rebalance my exam plan” or “why is tomorrow so busy?” while invoking the same planner backend rather than generating an untracked answer in chat.

21.6 Future adaptive planning

Over time, planner revisions may incorporate observed completion patterns, practice performance, or repeated delays. This should remain transparent and user-controllable; adaptation should never silently rewrite the plan without a visible revision.

22 Safety and Reliability

- Do not claim a plan guarantees mastery, a grade, or exam success.
- Clearly distinguish scheduling guidance from official syllabus/exam requirements.
- Preserve user-provided scope; do not silently insert or remove official content.
- Avoid unhealthy workload escalation. Respect configured daily limits and recovery time.
- Do not penalize users for missed work through shame-oriented language or loss framing.
- Replanning must be reversible before commit and should preserve completed history.
- AI failures must degrade to explicit retry/error states, not partial quest creation.
- Sensitive free-text inputs should follow existing Sylf data handling and logging policies.

23 Performance Requirements

- Mode selection and local form steps should feel immediate and must not wait on AI calls.
- Generation should display real orchestration stages if it takes noticeable time.
- Large syllabus inputs should be normalized/chunked server-side rather than blocking the browser.
- Avoid regenerating the entire plan when only one segment needs expansion or repair.
- Plan detail views should use bounded payloads; roadmap expansion should be paginated/lazy where appropriate.
- Rebalance should reuse existing normalized scope and completed state where possible.
- Quest commit should be transactional/idempotent and should not trigger duplicate downstream side effects.

Suggested product targets for MVP validation (not contractual SLOs): first visible generation progress within 500 ms after request acknowledgement; draft generation typically within a user-tolerable interactive window; navigation and editing remain independent of generation latency.

24 Analytics and Telemetry

Telemetry should measure whether users can successfully create, accept, execute, and adapt plans - not merely whether they open the feature.

Event / metric	Purpose
planner_mode_selected	Understand demand by planning intent.
planner_generation_started / completed / failed	Reliability and latency funnel.
planner_draft_edited	Measures how often users need to correct generated plans.
planner_committed	Primary activation event.
planner_quest_completion	Execution quality for planner-linked quests.
planner_rebalance_requested / applied	Measures adaptation need and usefulness.
planner_abandoned_step	Detect friction in guided input flow.
planner_feasibility_warning	Understand how often goals exceed available capacity.

Privacy rule: telemetry should not include raw syllabus text, exam notes, prompts, generated rationales, or private study content unless a separately reviewed analytics policy explicitly allows it. Prefer IDs, counts, mode, timing, and coarse categories.

25 Accessibility Requirements

- Entire guided flow and plan review must be keyboard accessible.
- Inputs require programmatic labels, descriptions, and error associations.
- Timeline/roadmap information must not rely on color alone.
- Drag-and-drop editing, if introduced, must have keyboard and non-drag alternatives.
- Dynamic generation/rebalance status must be announced appropriately without excessive live-region noise.
- Focus order must remain predictable as optional fields expand/collapse.
- Date selection must support keyboard entry and readable formatted dates.
- Reduced-motion settings should apply to transitions and progress visualization.
- Mobile controls must meet practical touch target sizing.

26 Functional Requirements Summary

ID	Requirement
FR-01	The user can choose Exam, Roadmap, or Mastery mode before generation.
FR-02	Each mode uses its own required inputs, granularity, and planning constraints.
FR-03	Exam mode works backward from a required exam date and supports multiple quest types.
FR-04	Roadmap mode defaults to month/week milestones and does not generate full-horizon daily quests.
FR-05	Mastery mode follows an understanding-to-mastery sequence with practice and review.
FR-06	Every generated plan is validated and presented as a draft before quest creation.
FR-07	The user can edit, lock, remove, and regenerate plan items before commit.
FR-08	Approved actionable plan items can create linked Sylf quests without duplicating quest logic.
FR-09	Rebalancing preserves completed items and respects locked items.
FR-10	Rebalancing presents a change preview before applying the revision.
FR-11	Planner state supports revisions and plan lineage.
FR-12	Generation and commit operations are authorized, validated, and idempotent.
FR-13	Existing Focus, Quests, themes, and auth remain authoritative.
FR-14	The planner exposes explicit feasibility warnings when scope cannot fit the available capacity.

27 Acceptance Criteria

27.1 Mode-level acceptance

☐ Exam Planner produces a deadline-bounded day-level plan with coverage, consolidation, practice, and readiness behavior where feasible.

☐ Exam Planner supports all defined quest types and preserves recovery capacity unless the user explicitly chooses a compressed plan.

☐ Roadmap mode produces month/week milestones and does not create hundreds of distant daily quests.

☐ A roadmap milestone can be expanded into a more detailed plan without rebuilding the entire roadmap.

☐ Mastery mode includes understanding, recall, practice, mistake review, and a mastery check unless the user explicitly edits the draft.

27.2 Shared acceptance

☐ The guided input flow is usable without exposing all optional fields at once.

☐ User can review and edit a generated draft before mutation of quests.

☐ Invalid/malformed AI output cannot be committed.

☐ Repeated commit requests do not create duplicate quests.

☐ Rebalancing never changes completed quest history.

☐ Rebalancing honors locked plan items and blocked study days.

☐ The planner integrates with existing Quest and Focus behavior rather than duplicating it.

☐ Plans correctly respect timezone/local dates.

☐ Existing themes/modes and responsive behavior are supported.

☐ Generation failures provide recoverable UI and do not leave partial persistent state.

☐ Telemetry excludes raw private syllabus/prompt content by default.

28 Phased Implementation Roadmap

28.1 Phase 0 - Architecture and contracts

- Audit existing AI Planner, Quest, Focus, auth, quota, and streaming/progress patterns.
- Finalize planner entities, structured AI output schema, revision semantics, and quest-link strategy.
- Define mode-specific validation rules and feasibility policy.
- Create test fixtures for representative exams, roadmaps, mastery plans, infeasible plans, and rebalances.

28.2 Phase 1 - MVP generation and review

- Mode selector and guided input flows.
- Exam, Roadmap, and Mastery generation.
- Structured draft review/edit UI.
- Deterministic validators and retry/repair flow.
- Commit approved actionable items into existing Quests.
- Plan detail/progress view.
- Basic analytics and reliability telemetry.

28.3 Phase 2 - Rebalancing and deeper integration

- Rebalance based on missed quests, availability changes, and progress changes.
- Revision diff preview and rollback-to-previous-draft where appropriate.
- “Plan this month” expansion from roadmap milestones.
- System/AI conversational commands backed by planner actions.
- More robust availability integration.

28.4 Phase 3 - Adaptive intelligence

- Performance-informed practice allocation.
- Mastery confidence based on user signals and completed practice.
- Smarter duration personalization based on observed study behavior.
- Optional document/syllabus import with explicit confirmation of parsed scope.
- Institution/exam-format templates where data quality and licensing permit.

29 Open Product Decisions

1. Should planner-created quests be automatically added after approval, or should the user choose which subset to commit?
2. What is the simplest availability model for MVP: hours/week, per-day availability, or reuse of an existing Sylf schedule?
3. Should confidence be a 3-level state (weak/okay/strong) or a 5-point scale?
4. How much editing should be supported in draft review: direct drag/drop, date picker, duration editing, locking, or all of these?
5. Should Roadmap mode use calendar months, arbitrary weeks, or both depending on horizon?

6. What is the product policy for infeasible plans: automatically create a compressed “triage” plan, or require explicit user opt-in?
7. Should the planner create explicit buffer/catch-up quests or leave capacity visibly unallocated?
8. Which existing Quest fields are sufficient for planner linkage, and which planner-specific metadata requires a separate entity?
9. How visible should AI-generated rationale be by default?
10. When should System proactively suggest a rebalance versus waiting for user action?

30 Technical Risks

Risk	Impact	Mitigation
Malformed AI output	Broken plan or unsafe quest commit	Strict schema validation, deterministic validators, repair/retry, no commit from free-form text.
Duplicate quest creation	User data corruption/confusion	Idempotency keys, transaction boundaries, stable planner item IDs.
Planner/quest state drift	Progress shown incorrectly	Derive from linked quest state where possible; versioned revisions.
Overloaded schedules	Poor user trust / unhealthy UX	Capacity limits, feasibility checks, explicit buffers and warnings.
Large syllabus cost/latency	Slow generation and high model cost	Normalize/chunk scope, bounded structured output, expand roadmap segments on demand.
Rebalance race conditions	Lost edits or overwritten revisions	Optimistic concurrency using current revision/version.
Timezone date shifts	Quests on wrong day	Canonical user timezone, date-only semantics where appropriate, dedicated tests.
AI invents official content	Misleading academic coverage	Treat user scope as authoritative; external enrichment only when explicitly enabled/verified.
Overly complex onboarding	Planner abandonment	Progressive disclosure, reuse known context, minimal required fields.

31 Recommended MVP Scope

The MVP should be intentionally narrower than the full vision. Recommended scope:

- All three modes available at launch.
- Text/paste-based scope input; file upload/document parsing can wait.
- Simple availability model with optional blocked days.
- Draft generation with structured schema and deterministic validation.
- Review/edit of title, date, duration, lock, remove, and regenerate segment.
- Exam stages and quest types as defined in this PRD.
- Roadmap milestone output plus “break down this milestone”.
- Mastery sequence output.
- Explicit approval before quest creation.

- Linked planner metadata on created quests.
- Manual “Rebalance” action using remaining plan state.
- No automatic performance-based adaptation in MVP.
- No automatic external syllabus lookup in MVP.

MVP success definition

A user can choose the correct planning intent, provide minimal academic scope and availability, receive a structurally valid plan, understand and edit it, commit it to Sylf Quests, execute those quests through existing Focus flows, and later rebalance the remaining work without duplicate or corrupted state.

32 Phase 2 / Future Enhancements

- File/PDF syllabus import with confirmation of parsed topics.
- Calendar and school timetable integration.
- Adaptive difficulty and practice allocation using performance signals.
- Automatic weak-area recovery plans.
- Exam-board or curriculum templates with verified source metadata.
- Question-generation or question-bank integrations for practice stages.
- Post-mock mistake analysis feeding the next revision.
- “What if?” planning scenarios (e.g., fewer hours/week, earlier completion target).
- Collaborative plans for study groups or Focus Parties.
- Teacher/tutor review links without exposing private unrelated user data.
- Plan templates users can save and reuse.
- Cross-subject workload balancing across multiple simultaneous exams.
- Predictive rebalancing suggestions based on repeated missed days, always requiring user approval before schedule mutation.

33 Appendix A - Example End-to-End Exam Flow

1. User opens AI Planner and selects **Prepare for an Exam**.
2. User enters “A-Level Physics Paper 4”, exam date, and pastes four syllabus topics.
3. Sylf asks for preparation level and uses the user’s selected study availability.
4. Server validates dates and calculates available capacity.
5. AI generates a structured draft using coverage, consolidation, practice, and readiness stages.
6. Deterministic validators confirm capacity, dates, quest types, and scope coverage.
7. User reviews the plan, moves one topic, locks a mock-exam date, and removes one redundant review.
8. User presses **Create Plan**.
9. Backend transactionally persists the active revision and creates linked quests.
10. Dashboard/Quests surfaces the next task; user starts a Focus session from it.
11. Two days later the user misses a quest and chooses **Rebalance**.
12. Sylf proposes a revision, showing what moves and how much buffer remains.
13. User approves; future quests update without changing completed history.

34 Appendix B - Product Decision Principles

When implementation choices are ambiguous, prefer the option that:

1. preserves user control over generated plans;
2. uses existing Sylf execution primitives instead of creating parallel systems;
3. makes the plan easier to scan and act on, not merely more detailed;
4. reduces long-horizon false precision;
5. keeps academic scope traceable to user-provided or explicitly verified sources;
6. fails closed on malformed AI output and data mutations;
7. allows future adaptation without making the MVP architecturally disposable.

End of PRD - Sylf AI Planner Modes - Draft v1.0